



UML

Lenguaje unificado de modelado

QUE ES UML ?

- **Language (Lenguaje):** No es un lenguaje de programación (como C#), sino un lenguaje visual. Se lee con los ojos, no con el compilador.
- **Unified (Unificado):** Antes cada programador dibujaba los sistemas como quería. UML unificó los símbolos para que un programador en Argentina, uno en Japón y uno en Microsoft entiendan lo mismo al ver un dibujo.
- **Modeling (Modelado):** Es crear una representación simplificada de la realidad. No dibujamos el código entero, dibujamos solo las "piezas" (clases) y cómo se conectan.

PARA QUÉ SIRVE EN UNITY ?

En el desarrollo de juegos, es muy fácil terminar con lo que llamamos "Código Espagueti" (scripts todos enredados entre sí). UML nos sirve para:

- **Planificar el sistema:** Saber quién conoce a quién antes de crear los scripts.
- **Comunicar ideas:** Si trabajás en un equipo, le pasás el diagrama a otro programador y él ya sabe qué variables y métodos tiene que crear sin que le expliques nada.
- **Documentar:** Si dejás el proyecto 6 meses y volvés, el diagrama te recuerda cómo funcionaba todo sin tener que leer 50 scripts de nuevo.

DIAGRAMA DE CLASE

Un diagrama de clase muestra las clases del sistema y relaciones. Describe la vista estática de un sistema.

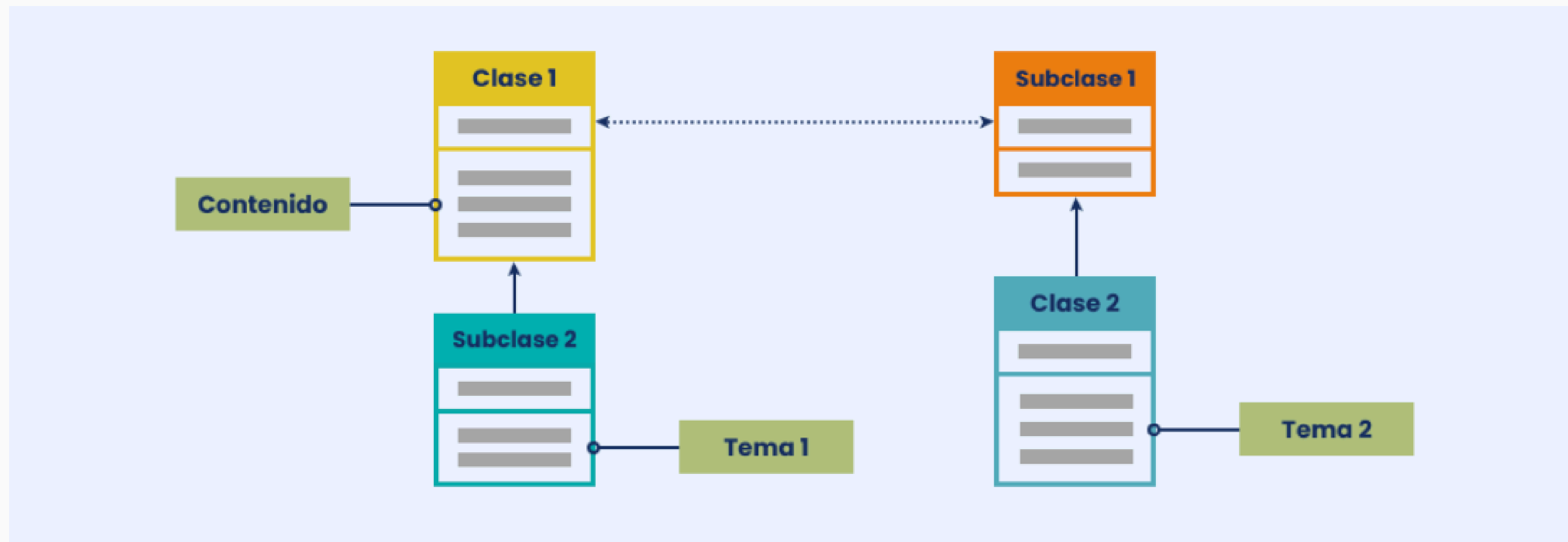


DIAGRAMA DE CLASE

1. Mermaid.js

No es una página en sí, sino un lenguaje que muchas páginas usan. Escribís texto y se transforma en diagrama al instante.

Página: [Mermaid Live Editor](#)

Por qué usarla: Podés guardar el código en un archivo .txt y reconstruir el diagrama cuando quieras.

2. Draw.io (Diagrams.net)

Es el estándar de la industria por ser gratuito y muy completo.

Página: [app.diagrams.net](#)

Por qué usarla: Tiene una biblioteca específica de UML 2.5. Podés arrastrar las clases, las flechas de agregación (diamante vacío) y composición (diamante lleno) muy fácil. Se integra con Google Drive.

3. PlantUML

Es el "abuelo" de los diagramas por código. Es extremadamente potente.

Página: [PlantText](#)

Por qué usarla: Es muy rápido para clases grandes. Si tenés que definir 20 scripts de Unity y sus relaciones, es mucho más veloz que andar arrastrando cajitas con el mouse.

4. Lucidchart

Es una opción más profesional y visualmente "pulida".

Página: [Lucidchart](#)

Por qué usarla: Tiene una función genial: podés importar código de C# y, en algunos casos, intenta generarte el diagrama solo (aunque la versión gratuita es limitada).

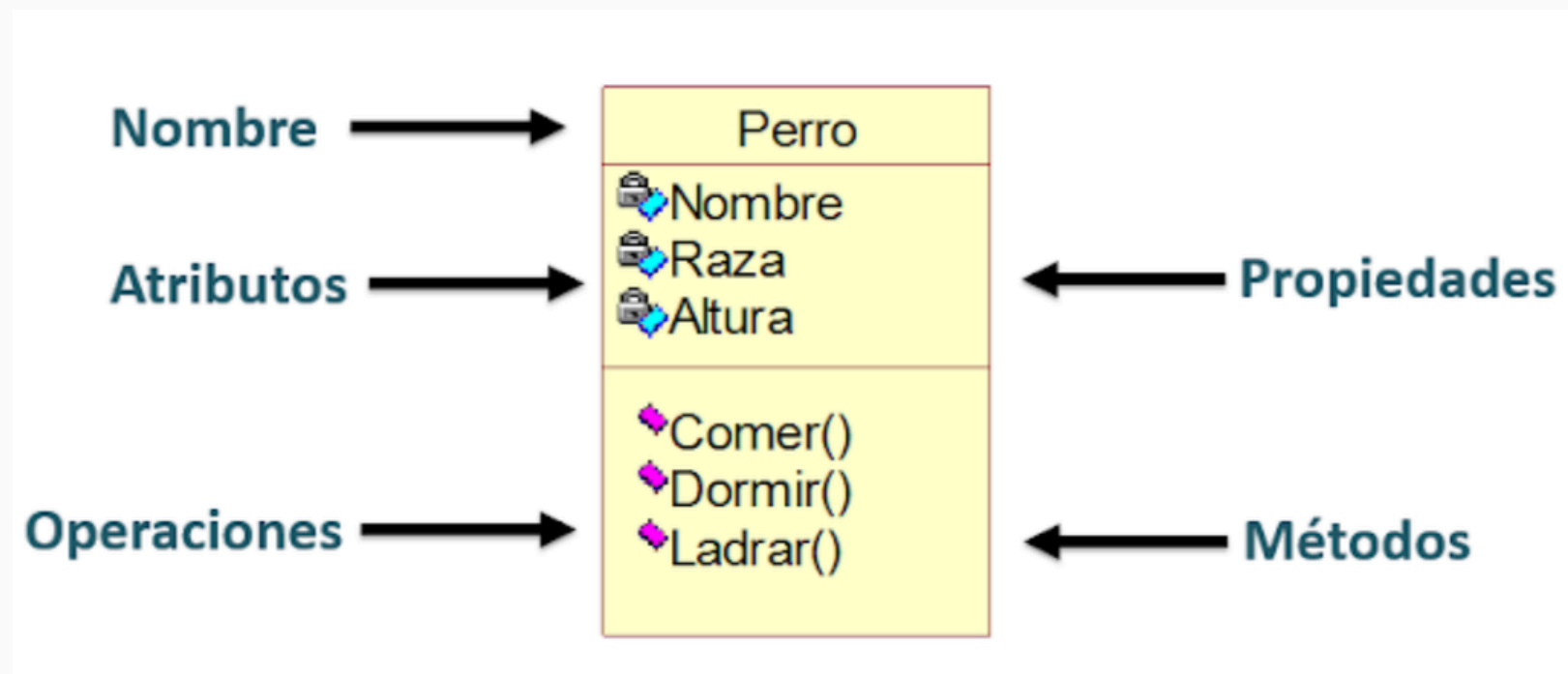
QUE ES UNA CLASE?

Una clase es una descripción de un conjunto de objetos que comparten los mismos atributos (estado), operaciones (mensajes), relaciones y semántica.

Variables en Unity

(Público): La variable es accesible desde otros scripts. En Unity, se ve automáticamente en el Inspector.

(Privado): Solo la clase puede usarla.



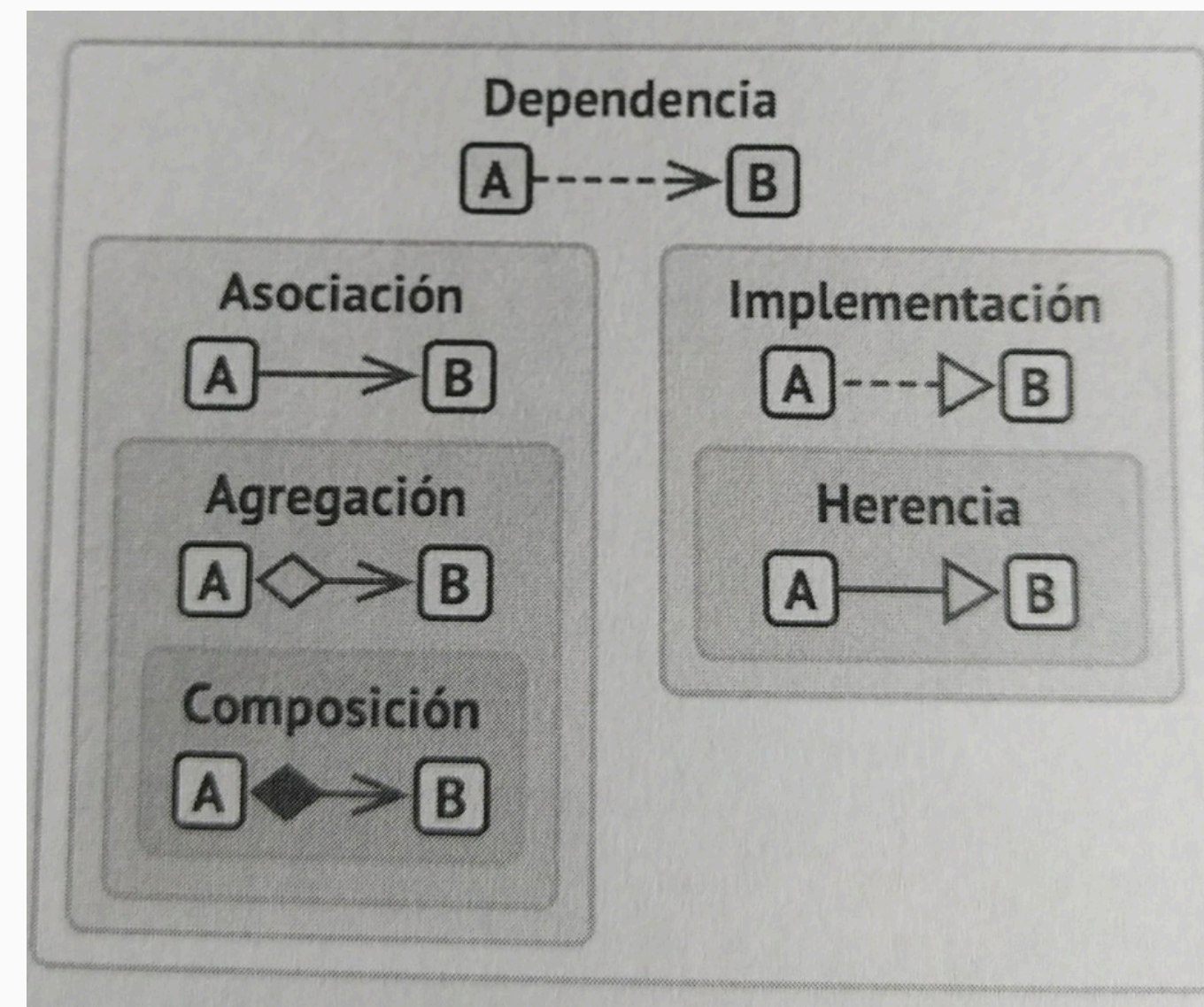
Tip Unity: No aparece en el Inspector, a menos que uses [SerializeField] antes de la variable.

¿Qué son las relaciones entre clases?

Una relación es una conexión semántica entre objetos. Proveen un camino de comunicación entre ellos.

Existen los siguientes tipos de relaciones

- Asociación
- Implementación
- Dependencia



Dependencia: La clase A puede verse afectada por los cambios en la clase B.

Asociación: El objeto A conoce el objeto B. La clase A depende de B.

Agregación: El objeto A conoce el objeto B y consiste en B. La clase A depende de B.

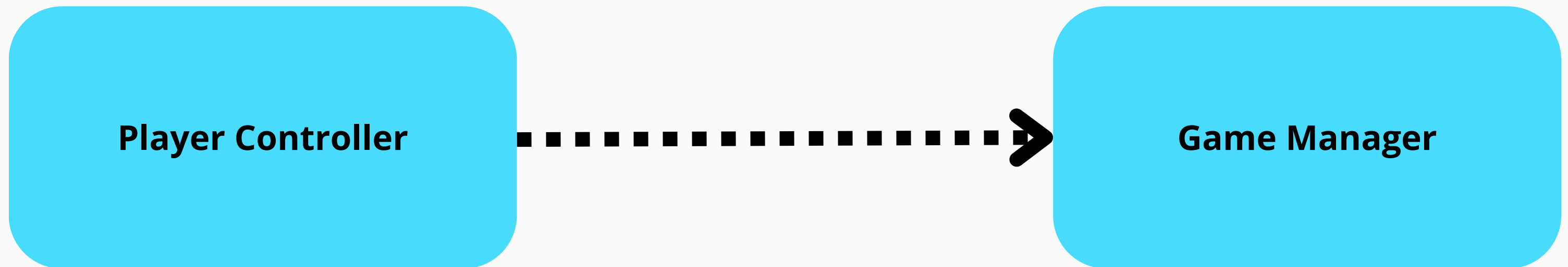
Composición: El objeto A conoce el objeto B, consiste en B y gestiona el ciclo de vida de B. La clase A depende de B.

Implementación: La clase A define metodos declarados en la interfaz B. Los objetos A pueden tratarse como B. La clase A depende de B.

Herencia: La clase A hereda la interfaz y la implementación de la clase B, pero puede extenderla. El objeto A puede tratarse como B. La Clase A depende de B.

Dependencia

La dependencia es el tipo de relación mas basica y debil entre clases. Existe una dependencia entre dos clases cuando ciertos cambios en la definicion de una clase puede provocar modificaciones en otra. La dependencia ocurre normalmente cuando utilizas nombre de clases concretas en tu codigo.



Por ejemplo, supongamos que tienes dos objetos: un "**PlayerController**" y un "**GameManager**", y el "**PlayerController**" necesita acceder a algunas funciones o variables del "**GameManager**"

```
1  using UnityEngine;
2
3  public class PlayerController : MonoBehaviour
4  {
5      private GameManager gameManager;
6
7      private void Start()
8      {
9          gameManager = GameObject.FindObjectOfType<GameManager>();
10     }
11
12     private void OnTriggerEnter(Collider other) {
13
14         if (other.CompareTag("Recollectable")) {
15
16             gameManager.IncrementScore();
17             Destroy(other.gameObject);
18         }
19     }
20 }
21
22
23
```

Logrando así que "**PlayerController**" dependa "**GameManager**" y se verá afectado en el caso de que GameManager sea modificado.

Una **Dependencia** es cuando una clase necesita a otra para realizar una tarea específica, pero no la guarda como parte de su estructura. Es como pedirle una herramienta prestada a un vecino: la usás y se la devolvés, no vive en tu casa.

Ventajas

- **Bajo Acoplamiento:** Los cambios en una clase afectan mínimamente a la otra.
- **Limpieza:** No ocupas memoria guardando referencias que solo usás una vez cada tanto.

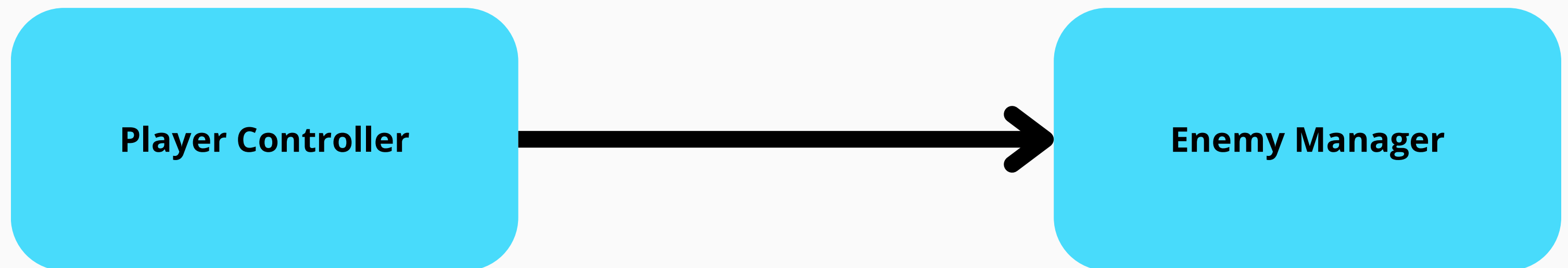
Desventajas

- **Fragilidad:** Si cambias el nombre de un método en la Clase B, la Clase A no se entera hasta que intentas ejecutar esa acción y el juego tira error.

*Uso típico en Unity: * UI Dinámica: Un script de Moneda que al ser recolectada busca al ScoreManager solo para sumar puntos: `FindObjectOfType<ScoreManager>().AddPoint()`. No guarda al manager, solo lo "usa".*

Asociación

La asociación es una relación en la que un objeto utiliza o interactúa con otro. La asociación puede verse como un tipo de especializado de dependencia, en la que un objeto siempre tiene acceso a los objetos con los que interactúa, mientras que la dependencia simple no establece un vínculo permanente entre los objetos.

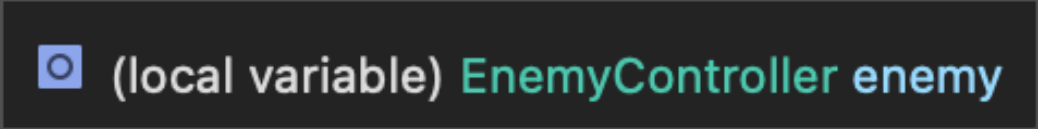


Por ejemplo, supongamos que el objeto "Player" tiene un script llamado "PlayerController" y el objeto "Enemy" tiene un script llamado "EnemyController".


```

1  using UnityEngine;
2
3  public class PlayerController : MonoBehaviour
4  {
5
6      private void OnTriggerEnter(Collider other)
7      {
8          if (other.CompareTag("Enemy"))
9          {
10             EnemyController enemy = other.GetComponent<EnemyController>();
11             if (enemy != null)
12             {
13                 enemy.AttackPlayer();
14             }
15         }
16     }
17 }
18

```

 (local variable) **EnemyController** enemy

En este ejemplo, cuando el objeto "**Player**" colisiona con un objeto que tenga el tag "**Enemy**", se llama al método **AttackPlayer()** del script "**EnemyController**" asociado al objeto "**Enemy**" que colisionó con el jugador. Esto establece una relación de asociación entre el objeto "**Player**" y el objeto "**Enemy**", donde el jugador puede interactuar con los enemigos mediante la colisión.

Asociación es cuando una clase tiene una referencia constante a otra. Es como tener el número de alguien en tu agenda: no son la misma persona, pero sabés cómo llamarlo en cualquier momento.

Ventajas

Velocidad: Como ya tenés la referencia guardada (en una variable), la comunicación es instantánea.

Facilidad: En Unity, es tan simple como arrastrar un objeto a una ranura del Inspector.

Desventajas

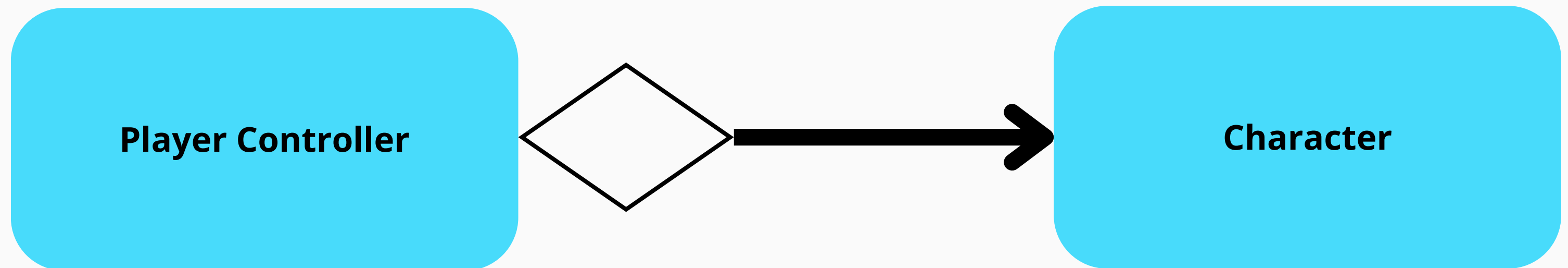
Dependencia de Escena: Si el objeto asociado no está en la escena, el script tira el famoso error *NullReferenceException*.

Uso típico en Unity: Cámaras y Seguidores: El script de SmoothCamera que tiene una variable public Transform target. La cámara conoce al jugador todo el tiempo para seguirlo.*

Agregación

La agregación es un tipo especializado de asociación que representa relaciones “uno a muchos”, “muchos a muchos” o “todo a parte” entre múltiples objetos.

Normalmente, con la agregación, un objeto tiene un grupo de otros objetos y sirve como contenedor o colección. El componente puede existir sin el contenedor y puede vincularse a varios contenedores al mismo tiempo.



Queremos modelar una relación de agregación donde un personaje puede contener múltiples objetos en su inventario.

```
1 using System.Collections;
2 using System.Collections.Generic;
3 using UnityEngine;
4
5 public class Item : MonoBehaviour
6 {
7     public string itemName;
8 }
9
10
```

```
1 using System.Collections;
2 using System.Collections.Generic;
3 using UnityEngine;
4
5 public class Character : MonoBehaviour
6 {
7     public List<Item> inventory = new List<Item>();
8
9     public void AddItemToInventory(Item item)
10    {
11        inventory.Add(item);
12        Debug.Log(item.itemName + " added to inventory.");
13    }
14
15    public void RemoveItemFromInventory(Item item)
16    {
17        if (inventory.Contains(item))
18        {
19            inventory.Remove(item);
20            Debug.Log(item.itemName + " removed from inventory.");
21        }
22        else
23        {
24            Debug.LogWarning(item.itemName + " is not in inventory.");
25        }
26    }
27
28    public void DisplayInventory()
29    {
30        Debug.Log("Inventory:");
31        foreach (Item item in inventory)
32        {
33            Debug.Log("- " + item.itemName);
34        }
35    }
36 }
37
38
39
```

```
public class PlayerController : MonoBehaviour
{
    void Start()
    {
        Character playerCharacter = gameObject.AddComponent<Character>();

        Item sword = new GameObject("Sword").AddComponent<Item>();
        sword.itemName = "Sword";
        playerCharacter.AddItemToInventory(sword);

        Item potion = new GameObject("Potion").AddComponent<Item>();
        potion.itemName = "Health Potion";
        playerCharacter.AddItemToInventory(potion);

        playerCharacter.DisplayInventory();
    }
}
```

En este ejemplo, la clase **Item** representa un objeto en el juego con atributos como el nombre del objeto. La clase **Character** representa un personaje en el juego con una lista de objetos (inventory) que contiene los objetos que el personaje tiene en su inventario. La clase PlayerController simula la creación de un personaje para el jugador y agrega algunos objetos a su inventario para luego mostrarlos.

Esta relación de agregación permite que los personajes del juego tengan sus propios inventarios de objetos, donde pueden agregar, quitar y mostrar los objetos que poseen.

Agregación es una relación de "todo a parte" donde las partes son independientes. Es como una carpeta con archivos: si borrás la carpeta, los archivos pueden seguir existiendo en el disco duro.

Ventajas

- **Reutilización:** Los objetos contenidos (como los items) pueden pasar de un dueño a otro.
- **Flexibilidad:** Podés tener una lista vacía o con mil objetos sin cambiar la lógica de la clase.

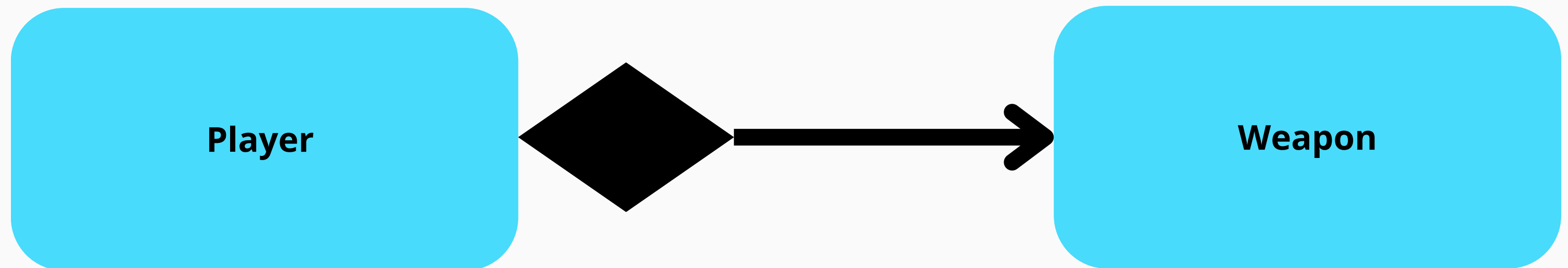
Desventajas

- **Gestión de Listas:** Requiere programar con cuidado el agregar y quitar elementos (List.Add, List.Remove) para no tener errores de conteo.

Uso típico en Unity: Inventarios: Un Personaje que tiene una List<Item>. Si el personaje muere y desaparece, podés programar que los items se queden tirados en el suelo (no mueren con él).*

Composición

La composición es un tipo específico de agregación en la que un objeto se compone de una o más instancias del otro. La diferencia entre esta y otras relaciones está en que el componente solo puede existir como parte del contenedor.



Un jugador que tiene un arma como parte de su composición. Cuando el jugador es destruido, el arma también debe ser destruida.

```

using UnityEngine;

public class Player : MonoBehaviour
{
    public Weapon weapon;

    void Start()
    {
        GameObject weaponObject = new GameObject("Weapon");
        weapon = weaponObject.AddComponent<Weapon>();
        weapon.damage = 10;

        weaponObject.transform.parent = transform;
    }

    void Update()
    {
        if (Input.GetKeyDown(KeyCode.Space))
        {
            if (weapon != null)
            {
                weapon.Attack();
            }
        }
    }

    public void DestroyPlayer()
    {
        Destroy(gameObject);
    }
}

```

```

public class Weapon : MonoBehaviour
{
    public int damage;

    public void Attack()
    {
        Debug.Log("Weapon deals " + damage + " damage!");
    }
}

```

Esto ejemplifica la relación de composición, donde el jugador es responsable de la creación y gestión del arma, y el arma existe solo mientras exista el jugador.

Composición Es la relación más fuerte. El objeto hijo es una pieza fundamental del padre y su existencia depende totalmente de él. Es como las páginas de un libro: si quemás el libro, las páginas desaparecen con él.

Ventajas

- **Seguridad:** Te asegurás de que no queden objetos "suelos" o basura (Memory Leaks) en el juego.
- **Consistencia:** El padre tiene control total sobre cuándo nace y muere su parte.

Desventajas

- **Rigidez:** Es difícil separar las partes una vez creadas. Si querés que un "brazo" de un robot pase a otro, la composición te obliga a hacer lógica extra de destrucción y recreación.

Uso típico en Unity Jerarquías Críticas: Un EnemyBoss que tiene una IA_Brain y un SistemaDeVuelo. Si destruí al Boss con Destroy(gameObject), esos componentes mueren con él porque son parte de su estructura física y lógica.

UNITY TIPS

El peligro del "Código Acoplado" (Asociación)

Si la Clase A conoce a la B, la B a la C, y la C a la A, tenés un círculo vicioso. En Unity, esto hace que si borrás un objeto, el proyecto entero tire errores en cadena.

Consejo: Si dos objetos se conocen demasiado, quizás deberían ser uno solo o comunicarse mediante una Dependencia puntual en lugar de una Asociación permanente.

[SerializeField] vs Public (Visibilidad)

Esto mantiene la relación protegida en código pero permite al diseñador asignar la Asociación arrastrando objetos en el Inspector.

La "Dependencia" de los Componentes

Consejo: Nunca uses GetComponent dentro de un Update(). Es como pedirle la herramienta al vecino 60 veces por segundo. Hacé la búsqueda en el Awake o Start y guardala como una Asociación privada.

Composición y el método OnDestroy()

En la Composición, el dueño es responsable de la limpieza.

Consejo: Si usás Composición para crear un objeto mediante código (sin hacerlo hijo en la jerarquía), recordá usar OnDestroy() para eliminar manualmente la parte. Si no, ese objeto queda "flotando" en la memoria de Unity (Memory Leak).

Evitá el "Objeto Dios" (Asociación excesiva)

Consejo: Si una clase tiene más de 5 o 7 flechas de Asociación saliendo de ella, está haciendo demasiadas cosas. Dividila en clases más chicas.

Multiplicidad y Arrays Fijos

Consejo: Si sabés que un coche siempre tiene exactamente 4 ruedas, usá un Array fijo Rueda[4] en lugar de una List<Rueda>. Esto le dice a Unity y a otros programadores que la estructura es rígida y más eficiente.

Prefabs como Plantillas de Composición

Un Prefab es, en esencia, un diseño de Composición guardado en el disco.

Consejo: Cuando diseñas un Prefab (ej: un enemigo con sus armas y efectos), estás definiendo que esos componentes siempre nacerán y morirán juntos. Si algo del Prefab debería sobrevivir a él, no lo hagas parte del Prefab, maneja-lo como una Agregación externa.



CONCLUSIÓN

Si solo lo usás una vez: **Dependencia.**

Si lo necesitás consultar seguido: **Asociación.**

Si es algo que el objeto "*lleva*" pero puede soltar: **Agregación.**

Si es algo que el objeto "*es*" o "*forma parte de su cuerpo*": **Composición.**